

Sri Lanka Institute of Information Technology



IT3012 - Intelligent Agents Year 3, Semester 1- 2026

Practical Answer Submission

Practical Sheet No: 01

Student ID	IT24102223
Student Name	Ekanayaka S.R.W.M.S.S.B.
Campus/ Center name	Malabe
Specialization	Artificial Intelligence
Batch No	Y3.S1.WD.AI.0101

Task 1.1

Q1)

- Performance measure – the objective metric that evaluates the agent's success.
- Environment – the external world the agent operates in.
- Actuators – the mechanisms the agent uses to change the environment (actions).
- Sensors – the means by which the agent receives percepts from the environment.

Q2)

- self.width & self.height
 - Define the boundaries and spatial dimensions of the physical 2D grid environment.
- self.agent_pos
 - Represents the current physical location $[x, y]$ of the primary agent within the grid world.
- self.walls
 - Represents static spatial obstacles (coordinates of wall cells) in the environment.
- self.food_positions
 - Represents the positions of all remaining collectable food items located in the grid environment.
- self.opponents
 - Represents the coordinates $[x, y]$ of dynamic adversarial entities operating within the environment.

Q3) This environment is classified as a **Multi-Agent** environment.

Justification

1. **Multiple Autonomous Entities (self.opponents):** The initialization of self.opponents introduces multiple active entities into the environment alongside the primary agent.
2. **Independent Behavior:** As seen in

execute_action, opponent positions update autonomously each step as they choose their own actions (random.choice(['Up', 'Down', 'Left', 'Right', 'Stay'])).

3. **Adversarial / Competitive Dynamics:** The opponents' behavior directly affects the primary agent's performance and environment state. A collision with an opponent penalizes the agent (self.score -= 50) and triggers an immediate game-over condition (self.collision = True).

Task 1.2

Q4)

A **percept sequence** is the complete history of all percepts that an agent has received over its entire lifetime – i.e., the chronological list of every sensor reading from time 0 up to the current moment. It is the agent's full experiential record.

Q5)

The `get_percept()` method represents the **Sensors (\$\$\$)** component of the **PEAS** framework. Why Sensors (\$\$\$)?

In AI agent design:

- **Sensors (\$\$\$)** provide the agent with its percept sequence (information about the environment state at any given step).
- In `VisualGridHuntGame.get_percept()`, the method acts as the agent's sensory interface, capturing and returning sensory signals such as 'smells_food', 'hit_wall', 'agent_pos', and 'opponent_positions'.

Q6)

Classification

This environment is **Partially Observable**.

An environment is **Fully Observable** if an agent's sensors give it access to the complete state of the environment at any given time step. Because key aspects of the physical world state are missing from the percept dictionary in `get_percept()`, the agent has incomplete information.

What the Agent CAN See

- **Agent position:** Its exact current coordinates ('agent_pos').
- **Opponent positions:** Coordinates of all opponents ('opponent_positions').
- **Immediate local status:** Whether food is at its *current* square ('smells_food') or if it hit a wall at its *current* square ('hit_wall').
- **Aggregate metrics:** Total count of remaining food ('remaining_food'), score ('score'), and collision state ('collision').

What the Agent CANNOT See

- **Global Food Locations:** The agent **does not receive the coordinates of food items** across the grid (`self.food_positions` is omitted). It only knows if food is present at its exact current location (`smells_food`) and how many total food items remain.
- **Global Wall Layout:** The agent **does not receive the map of wall coordinates** (`self.walls` is omitted). It only learns it hit a wall after attempting a move into one (`hit_wall`).

Because the agent cannot see where food items or walls are located across the grid without exploring and remembering past percepts, the environment is **Partially Observable**.

Task 1.3

Q7)

The PEAS component being actively updated is the **Performance Measure (\$P\$)**.

Why Performance Measure (\$P\$)?

In the **PEAS** framework:

- **Performance Measure (\$P\$)** defines the metric used to evaluate how successfully the agent is performing its task.
- In `execute_action()`, deducting points (`self.score -= 5`) penalizes unwanted behavior (hitting a wall), directly updating the numeric score that quantifies the agent's overall success according to the performance criterion.

Q8)

It is structurally important for a **Performance Measure (\$P\$)** to evaluate changes in the external environment state rather than internal processing effort for several core reasons in AI system design:

1) Objective Evaluation vs. Agent "Self-Delusion"

A performance measure is designed from the **environment designer's perspective**, not the agent's.

- Evaluating external outcomes (e.g., hitting a wall vs. moving into an open cell) ensures success is judged by actual real-world results.
- If performance were based on internal processing effort (e.g., CPU cycles or nodes expanded), an agent could achieve a high score simply by "thinking hard" or spinning compute loops in place, even if its actual actions in the world are catastrophic (like continuously crashing into walls).

2) Separation of *What* (Objective) from *How* (Mechanism)

The PEAS framework strictly decouples **what** constitutes desirable behavior in the environment from **how** an agent chooses to compute it:

- The external environment metric (\$P\$) remains constant whether the agent is a **Simple Reflex Agent** (lookup table) or a complex **Search Agent** (A*/BFS).
- Judging internal effort would unfairly favor or penalize specific algorithmic implementations rather than evaluating task completion.

3) Preventing Goal Hacking and Reward Exploitation

If internal effort were tied to rewards/penalties:

- An inefficient or buggy algorithm that takes twice as long to make a move could be scored differently than an efficient one, distorting the measure of task success.
- Grounding penalties directly in physical environment interactions (such as hitting a wall in `execute_action()`) forces the agent to adapt its internal behavior to conform to physical constraints rather than manipulating its own metric.

Task 2.1

Q9)

If `self.toxic_traps` exists in the physical environment state but is omitted from `get_percept()`, it affects the environment's **Observability** classification in the following specific ways:

1. Reinforces and Deepens **Partial Observability**

- If an environment conceals any relevant state variable needed for decision-making, it cannot be fully observable.
- Because `self.toxic_traps` directly impacts the physical state and performance evaluation, keeping it hidden ensures the environment remains strictly **Partially Observable** (or creates an **unobserved state dimension** regarding hazards).

2. Introduces an **Unobservable Hazard / Latent State Variable**

- Unlike `self.walls` (where the agent receives feedback via 'hit_wall' when it bumps into one) or `self.food_positions` (where the agent receives 'smells_food' when on a food cell), hidden traps introduce a state component for which the agent has **zero immediate sensing capability** prior to stepping on them.
- This forces the agent to operate under **uncertainty**—it cannot deduce trap locations purely from its current percept, requiring it to rely on history/memory or probabilistic belief states to estimate safe paths.

Task 2.2

Q10)

A **Rational Agent** selects actions that maximize its **expected performance measure (expected utility)** given the information provided by its percept sequence and built-in knowledge.

Adding the 'smells_toxin' sensor in `get_percept()` enhances rationality in the following key ways:

1. Informs Expected Utility Calculations

Without 'smells_toxin', the agent has no perceptual feedback regarding whether its current state is hazardous. With 'smells_toxin', the agent gains explicit sensory data alerting it that its current position carries a negative expected utility (score penalty).

2. Enables Adaptive Risk Mitigation

- **Condition-Action Rules (Reflex):** The agent can implement immediate evasive rules (e.g., IF smells_toxin IS True THEN move_to_adjacent_safe_cell).
- **Internal World Model (Model-Based Agent):** A model-based agent can remember the coordinate associated with smells_toxin = True and mark it as a dangerous cell in its internal map, ensuring it avoids pathing through that coordinate in future steps.

3. Maximizes Cumulative Performance Measure (\$P\$)

By recognizing toxic cells upon entering them:

- The agent prevents **repeated penalties** (which would occur if it blindly lingered on or returned to unmapped toxic cells).
- Maximizing expected score over time requires trading off risks vs. rewards. The sensor provides the necessary information to compute and avoid low-utility actions, thereby maximizing overall expected utility.

Task 2.3

Q11)

The classic "**Vacuum World Exploit**" (from Russell & Norvig / Lecture 01 on Intelligent Agents) illustrates the danger of designing faulty performance metrics:

The Faulty Metric

If the performance measure is naively designed to give **1 point for every piece of dirt sucked up (or every "Clean" action performed)**:

The Agent's Exploitative Behavior ("Metric Hacking")

- Instead of keeping the environment clean, a rational agent optimizing this flawed metric learns to **maximally generate and suck up dirt repeatedly**.
- The agent could clean Square A (+1 point), deliberately dump/re-deposit dirt or wait for it to get dirty again, and repeatedly suck up the same dirt over and over to infinite point farming.

The Takeaway for AI Design

- **Rule:** Measure **environmental outcomes over time** (e.g., "*give 1 point for every square that is clean at each time step*"), rather than rewarding intermediate cleaning actions.
- **Relevance to Traps:** Similarly, if stepping on/exiting traps or moving around wasn't properly penalized, an agent could exploit state loops or infinite loops without achieving the actual goal of safely collecting food.